Project F: Random numbers from chaos — project kickstart

FMFN05 / FYST57 · Project, 1.5 credits · *Andrea Idini* [1]

2026-09-29

Why random numbers from chaos

Random numbers are everywhere in computational physics. The Monte Carlo method was born in Los Alamos in the 1940s with Ulam, von Neumann and Metropolis, and several projects of this course, like fire propagation and probabilistic cellular automata, use random numbers at every step. A pseudo-random generator is a deterministic map on a finite set of states started from a seed, while a true or physical generator samples a physical process, such as thermal noise.

A chaotic map produces information. As we've seen, the logistic map at $r = 4$ is conjugate to the tent map, its Lyapunov exponent is $\ln 2$, and the symbolic sequence $x_n < 1/2$ or $x_n > 1/2$ is a fair coin for typical initial conditions. This is what Ulam and von Neumann (1947) proposed as a random number generator. By Pesin's identity $h_{KS} = \lambda$, so a one-dimensional map produces at most $\lambda/\ln 2$ new bits per iteration. The catches are that the invariant density $\rho(x) = 1/(\pi\sqrt{x(1-x)})$ is not uniform, that taking more bits per iteration than the entropy allows gives correlated output, and that in finite precision every orbit ends on a cycle. The doubling map in double precision reaches 0 in about 50 iterations, and in single precision the logistic map has only seven cycles, the longest of period 487 (Persohn and Povinelli 2012). They all have linear recurrences over finite fields or integers, but are sometimes used because period, equidistribution and speed can be proven. However, chaos is used seriously in physical generators, where it amplifies microscopic noise: chaotic semiconductor lasers produce random bits at Gb/s rates (Uchida et al. 2008), and in these systems the entropy rate is set by the largest Lyapunov exponent while the noise supplies the seed (Mikami et al. 2012).

Careful that passing a test suite does not prove randomness. A test can only reject, and statistical randomness is not unpredictability. The Mersenne Twister passes almost all of TestU01, yet 624 consecutive outputs give away its whole state, and n symbols of the logistic map fix the initial condition to within 2^{-n}, after which every future bit is known.

So there is room here for several kinds of project: measure the entropy budget of a chaotic generator, compare its periods in finite precision with random-mapping statistics, build and validate the tests, simulate chaos as a noise amplifier, or break a chaotic generator by predicting it. All of these are welcome, and a question you arrive with yourselves is better than any of mine.

Guidelines

1.5 credits is about one full working week per person, and this project is meant for a group of three. That is enough for one clean idea, implemented and honestly analysed, plus a second one that tests the first. It is clearly not enough for a new cryptographic generator.

- Pick a question you can answer rather than a topic you can survey. "At how many bits per iteration does the logistic generator start failing the tests, and is that number $\lambda/\ln 2$?" makes a better project than a vague "Chaos-based random numbers".

[1] This document was drafted with the help of AI, but reviewed and modified by me

- Specifying and running one chaotic map through a test suite is not enough for three people. A reasonable scope combines a chaotic generator, a few tests you implement yourselves and validate on a good generator (PCG64) and a bad one (arguably yours), and a quantitative link to the dynamics (Lyapunov exponent, KS entropy, period) or an attack.
- Settle on day one how you turn iterates into bits, how many bits per iteration, the precision, and the reference generator.
- I suggest to get something running on day one, however crude. A map, a bit extractor, a histogram and a monobit test are enough to build on when you are on a deadline.
- Leave more than one day for analysis and writing. Generating bits is the quick part and the statistics are the slow part, and you will discover what you should have measured while writing.
- Validate before producing results. Your own tests should pass PCG64 and fail RANDU.
- Report what you implemented and measured rather than what you planned and hoped for.
- Read and respect the course's guidelines for the use of AI.

You can use whatever computational tools you have access and are comfortable with: python, MATLAB, Mathematica, Fortran… Test suites need long sequences. A chaotic map in pure python comparitively slow, so you might need to generate the bits in compiled or vectorised code and store and care about efficiency in order to use large test suites.

Ideas for project directions

The entropy budget. Take the logistic map at $r = 4$, the tent map and a Chebyshev map, and extract k bits per iteration, from the threshold at $1/2$ for $k = 1$ to the leading bits of the uniformised variable $y = (2/\pi) \arcsin \sqrt{x}$ for larger k. Find the k at which the tests start to fail, and compare with $\lambda / \ln 2 = 1$ bit per iteration. Estimate the entropy rate independently from block entropies of the symbolic sequence, and check whether the failure point follows λ when you change it, with the logistic map at $r < 4$ or the skew tent map. As an attack, reconstruct the seed from the first bits by backward iteration and predict the rest. Start from Ulam and von Neumann (1947).

Finite precision and periods. For a random map on N states the expected tail and cycle length of a random point are both about $\sqrt{\pi N/8}$ (Flajolet and Odlyzko 1990), about 1800 for $N = 2^{23}$. Persohn and Povinelli (2012) found in single precision a cycle of period 487 collecting most of the seeds, much shorter than this estimate. Enumerate all cycles and transients of the logistic map in fixed point with b bits for b up to about 25, plot the mean cycle length against $2^{b/2}$, and extend to float32 and, by sampling, to float64. Then try the usual fixes, perturbing the orbit or coupling several maps, and check whether they really lengthen the period. The encryption project has a related direction on finite precision.

Test the tests. Implement a few tests yourselves, for example monobit frequency, runs, block frequency, serial or block entropy, approximate entropy and the spectral test. Check that the p-values are uniform on PCG64 and that the tests catch RANDU, a small LCG and the raw bits of the logistic map. RANDU passes most one-dimensional tests, but its consecutive triples lie on only 15 planes. Measure the power of each test, that is how long a sequence it needs to reject a given defect, and compare with SmallCrush and Crush on the same generators. Which defect of a chaotic generator does each test see? Start from Rukhin et al. (2010) and L'Ecuyer and Simard (2007).

Chaos as a noise amplifier. Simulate the logistic map with additive noise of size ϵ, or a noisy

Lorenz flow, and sample one bit at intervals comparable to the Lyapunov time. A perturbation of size ϵ grows to the size of the attractor in about $\ln(1/\epsilon)/\lambda$, and after that the bit is no longer determined by the past. Measure the entropy of the bits conditioned on the previous ones as a function of the sampling interval and of ϵ, and check whether the entropy rate approaches λ. Post-process with the von Neumann extractor (01 and 10 give a bit, 00 and 11 are discarded) and compare the rate with the chaotic lasers of Uchida et al. (2008). Start from Mikami et al. (2012) and Stojanovski and Kocarev (2001).

You should have most conceptual and mathematical instruments to run this project directly from the course and from the literature, but ask if something is unclear.

Measuring what you build

You will meet these tests in almost every paper, so it is worth knowing them even if you only use a few.

- **Bit frequency and runs.** The fraction of ones should be $1/2$ within about $1/(2\sqrt{n})$ for n bits, and the number of runs that of a fair coin.
- **Block entropy.** The Shannon entropy of k-bit words should approach k bits, and the difference of consecutive block entropies estimates the entropy rate, to compare with $\lambda/\ln 2$.
- **Autocorrelation.** For independent bits $|C(\tau)|$ is of order $1/\sqrt{n}$ for $\tau \neq 0$, and the spectrum is flat.

Due to the statistical nature of randomness, it is possible that test fails just by sheer chance, so a single failure or pass means little and what matters is the distribution of p-values over many sequences. Also, testing the raw floats of the logistic map instead of extracted bits only reports its invariant density.

Deliverables

A report of a few pages, 4 to 10 depending on the prose, and your code, by the exam date. Say in the report who in the group did what.

Include figures of the histogram of the raw iterates against the invariant density, of the distribution of p-values, and of the quantitative measurement your project is about.

- Give every number an uncertainty and, where theory offers one, an expected value to compare against.
- State how bits are extracted, the precision, the seeds, the sequence lengths and the test parameters, so that someone could reproduce your numbers.
- Show at least one validation with a known answer, for example a good and a bad generator through your own tests.
- Write a short paragraph on the outlook: what can be developed further, what you did not test but should or would have.

References

You do not need all of these but only those related to your project.

- G. Ohlen, S. Aberg, P. Ostborn, *Chaos*, Division of Mathematical Physics, LTH (2007). The course book.
- S. M. Ulam, J. von Neumann, *On combination of stochastic and deterministic processes*, Bull. Amer. Math. Soc. 53, 1120 (1947). The original idea, a one paragraph abstract.
- T. Stojanovski, L. Kocarev, *Chaos-based random number generators, Part I: Analysis*, IEEE Trans. Circuits Syst. I 48, 281 (2001). https://doi.org/10.1109/81.915385 A noise-driven piecewise-linear map, with the entropy worked out.
- A. Uchida et al., *Fast physical random bit generation with chaotic semiconductor lasers*, Nature Photonics 2, 728 (2008). https://doi.org/10.1038/nphoton.2008.227 The start of chaotic-laser generators.
- T. Mikami et al., *Estimation of entropy rate in a fast physical random-bit generator using a chaotic semiconductor laser with intrinsic noise*, Phys. Rev. E 85, 016211 (2012). https://doi.org/10.1103/PhysRevE.85.016211 Where the randomness comes from.
- A. Rukhin et al., *A statistical test suite for random and pseudorandom number generators for cryptographic applications*, NIST SP 800-22 rev. 1a (2010). https://csrc.nist.gov/pubs/sp/800/22/r1/upd1/final The 15 tests, clearly described.
- P. L'Ecuyer, R. Simard, *TestU01: A C library for empirical testing of random number generators*, ACM Trans. Math. Softw. 33, 22 (2007). https://doi.org/10.1145/1268776.1268777 The standard battery.
- P. Flajolet, A. M. Odlyzko, *Random mapping statistics*, EUROCRYPT '89, LNCS 434, 329 (1990). https://doi.org/10.1007/3-540-46885-4_34 Expected cycle and tail lengths.
- K. J. Persohn, R. J. Povinelli, *Analyzing logistic map pseudorandom number generators for periodicity induced by finite precision floating-point representation*, Chaos Solitons Fractals 45, 238 (2012). https://doi.org/10.1016/j.chaos.2011.12.006 All the cycles of the logistic map up to single precision.